

Over-sampled Data Converters – ET4278

Practical Session 1

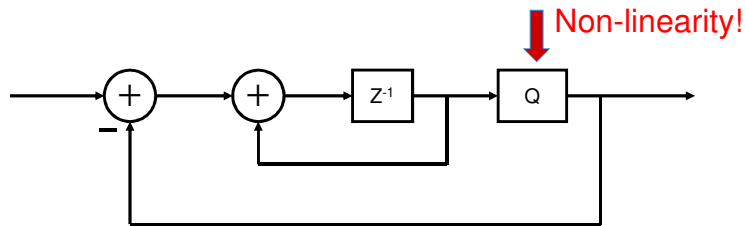
April 30th, 2019

Exercises & Assignments

During this session you will be learning by doing!

- You will be running and modifying Matlab m-files that simulate the behaviour of a 1st order $\Sigma\Delta$ modulator
- There are also **homework** assignments to be handed in. Make sure that you HIGHLIGHT YOUR modifications to the basic m-files.
- Note: The results of the homework assignments will contribute (about 30%) towards the final grade

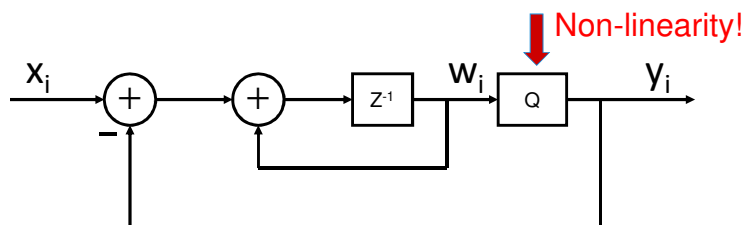
Time Domain Simulation (1)



A $\Sigma\Delta$ modulator is a fundamentally **non-linear** system

- So the linear model (quantizer error = white noise) is only an **approximation** (which provides useful insights)
- **Certainty** about the operation of a $\Sigma\Delta$ modulator
 $\Rightarrow$ time-domain simulations

Time Domain Simulation (2)



- Simulations are based on the modulator's difference equations:

$$w_i = w_{i-1} + x_{i-1} - y_{i-1}$$

$$y_i = Q(w_i)$$

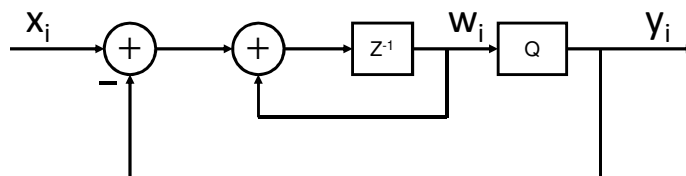
- For a single-bit quantizer:

$$Q(x) = 1, x \geq 0$$

$$Q(x) = -1, x < 0$$

- For simplicity, we will assume that the DAC just copies $Q(x)$ i.e. that it outputs ± 1 V signals

Matlab Simulation (1)



The main loop only requires a few lines of code!

```
for i=2:N                                     % Matlab indices start from "1"
    w(i) = w(i-1) + x(i-1) - y(i-1);          % Next output of accumulator
    y(i) = sign(w(i));                        % Next output of the quantizer
end;
```

Matlab Simulation (2)

```
for i=2:N
    w(i) = w(i-1) + x(i-1) - y(i-1);
    y(i) = sign(w(i));
end;
```

- FOR loops are **slow** because they require **serial** processing
- To make the loop run faster
 1. Preallocate arrays e.g.


```
w = zeros(1:N)
```
 2. Minimize the number of calculations in loops
 - Calculate the acc. input "e" after the FOR loop e.g. $e = x - y$;
 3. Perform vector computations as much as possible, e.g.


```
for i=1:N,
    e(i) = x(i) - y(i);  =>  e = x - y;
```

```
end;
```

Matlab Simulation (3)

```

N = 10;           % simulate 10 clock cycles

x = zeros(1,N); % preallocate: x(1) = x(2) = ... x(N) = 0
w = x; y = x;    % preallocate the other arrays

% main loop
for i = 2:N
    w(i) = w(i-1) + x(i-1) - y(i-1);
    y(i) = 2*(w(i) >= 0) - 1; % Why not Matlab's "sign" function?
end;

```

Challenge: come up with a faster implementation of "Q"

Matlab Simulation (4)

Plotting the results:

```

t = 0:N-2;           % t = 0,1,2, ... N-2
n = t+2;             % n = 2,3,4, ... N

plot(t,x(n),'k-');    % plot "x" with black lines
hold on;              % allows the current plot to be overwritten
plot(t,w(n),'r-');
plot(t,y(n),'b-');
hold off;             % good habit!

plot(t,x(n),'k-',t,w(n),'r-',t,y(n),'b-'); % alternative

```

NOTE: For help on the plot function type: **help plot**

Matlab Programming

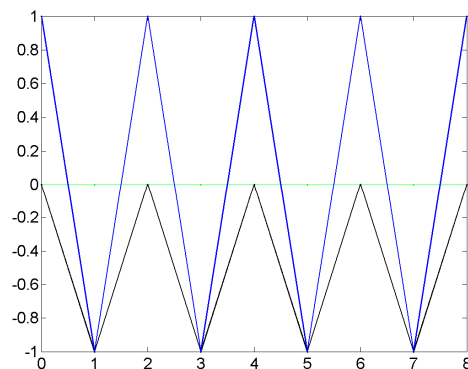
During this session you will be learning by doing! So DO!

- Before asking questions, use the “help” command!
- Write fast code i.e. preallocate and minimize in-loop computation
- Use a lot of comments so that we at least know what you were trying to do!
- Submit the homework assignments to the TA
- Always BEFORE the next class (no exceptions)
- Only one file: ready-to-run m file with comments

Exercise 1 (10min)

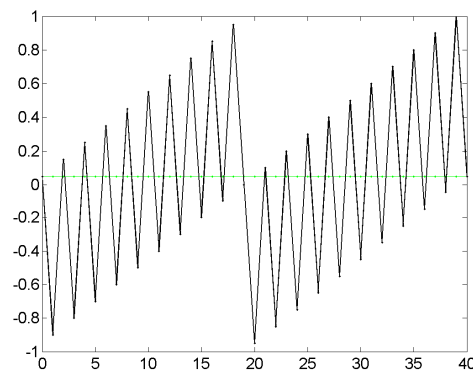
- Download (from Blackboard) and run the m-file: MOD1.m
 - This m-file simulates the behaviour of a 1st order modulator with a DC input $x = 0$
 - Go through the m-file, and identify its various sections
 - Observe the accumulator output w and the output bitstream y
- Apply DC inputs $x = 0.05V$, and $x = 0.3142V$ to the modulator.
 - Again observe the accumulator output and the bit-stream.
 - Compute the bitstream average, i.e. by adding a simple line of code: `average = sum(y)/length(y);`
 - Is the bitstream average equal to the input offset?
 - Increase the “nr_periods” from 10 to 1000. Observe the behavior of the bitstream average. What do you notice?

Input = 0



- Modulator output "idles" i.e. oscillates between 1 and -1
- With a frequency of $F_s/2$ = highest possible frequency

Input = 0.05



- $x = \text{zeros}(1,N) \Rightarrow x = 0.05 * \text{ones}(1,N)$
- Note: the modulator's output frequency changes!

Exercise 2 (10min)

- Download and run the m-file: MOD1_filt.m
 - This code implements a FIR filter after the modulator.
 - Go through the code, and identify the parts that define and implement the FIR filter.
 - The decimated output value at the output of each filter is shown after the Matlab command prompt.
 - Calculate the quantization error for each filter, with a simple line of code.
 - Study the filters in the time-domain and the filtered output bit-stream in the plotted graphs.
- Increase the length of the FIR filter
 - What happens to the quantization error of the different filters?
- For various input offset values between -1V and 1V compare the quantization error of the filters.

FIR-filter coefficients

Rectangular

(Sinc)

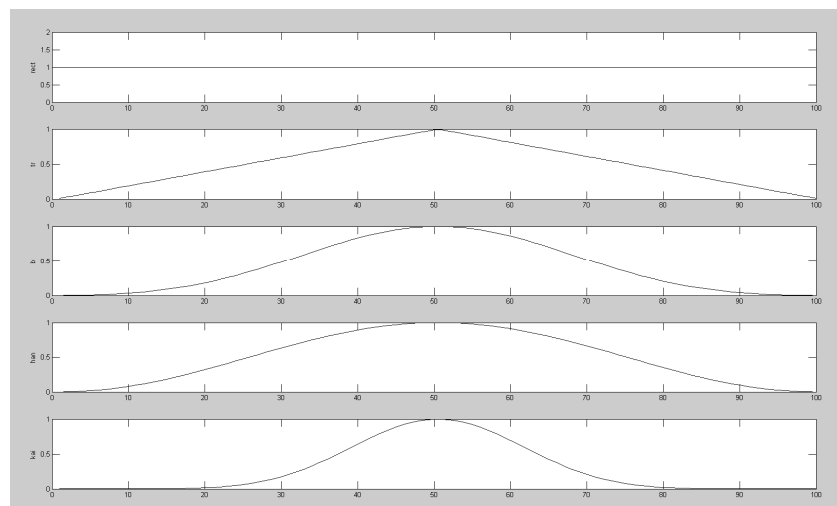
Triangular

(Sinc²)

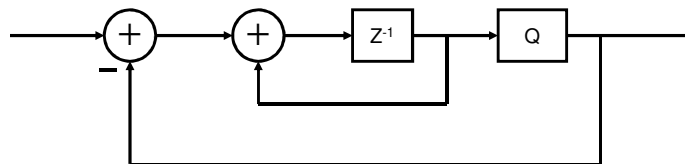
Blackman

Hann

Kaiser



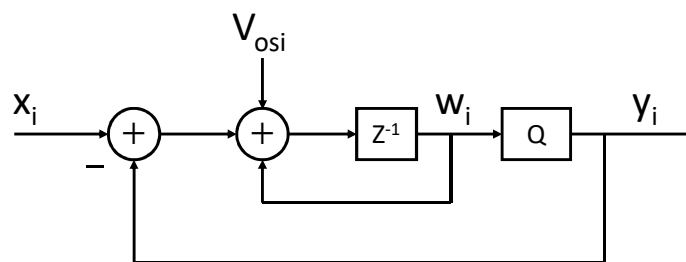
Time Domain Simulation



A $\Sigma\Delta$ modulator is a fundamentally **non-linear** system

- So simulations are needed
- However, the linear model (quantizer error = white noise) provides useful insights
- Question: according to the Linear Model what do we expect to happen to Accumulator offset and Quantizer offset?

Accumulator Offset

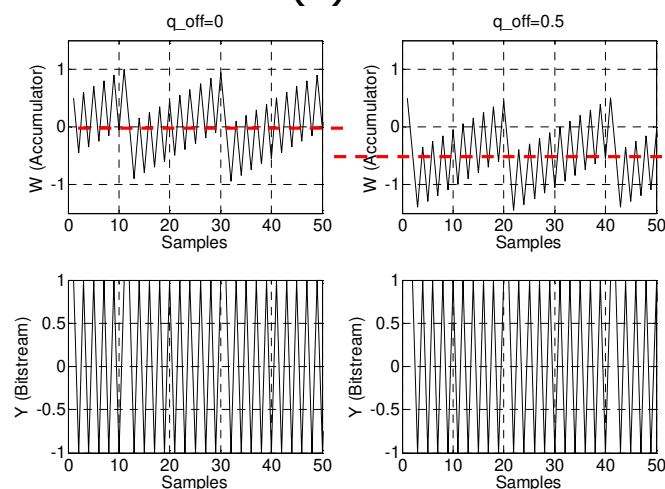


- Accumulator may suffer from **offset**
- Accumulator offset just shifts the input signal
- But what is the effect of quantizer offset?

Exercise 1.3 (10min)

- Run the m-file: MOD1_offset.m
 - There are two new variables in this file:
 - a_off : Accumulator Offset
 - q_off: Quantizer Offset
 - Run the code with these values equal to 0 and note the decimated output at the Matlab command line.
 - Observe the accumulator output and the bitstream output
- Apply 0.5V of offset to the Accumulator i.e. a_off = 0.5, q_off = 0
 - What happens to the decimated value?
 - What happens to the waveforms? Try other values...
- Make a_off = 0 and apply 0.5V of offset to the Quantizer (q_off)
 - What happens to the decimated value?
 - What happens to the graphs?

Quantizer Offset (1)

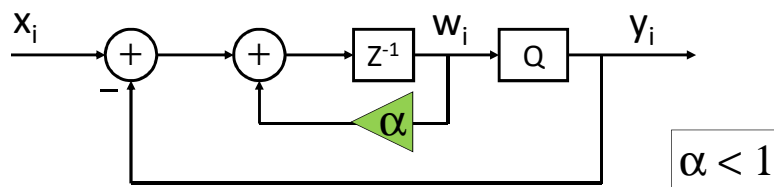


- $x = 50\text{mV}$ and quantizer **offset** = 0.5V
- Accumulator swing shifts by 0.5V, bitstream is the same??

Quantizer Offset (2)

- $\text{NTF} = 1 - z^{-1} \Rightarrow$ at DC $\text{NTF}=0$
- Quantizer offset is suppressed by the accumulator's gain just like quantization **noise**
- But Q-offset modifies integrator swing
- In general, quantizer non-idealities do not affect performance

Accumulator Leakage

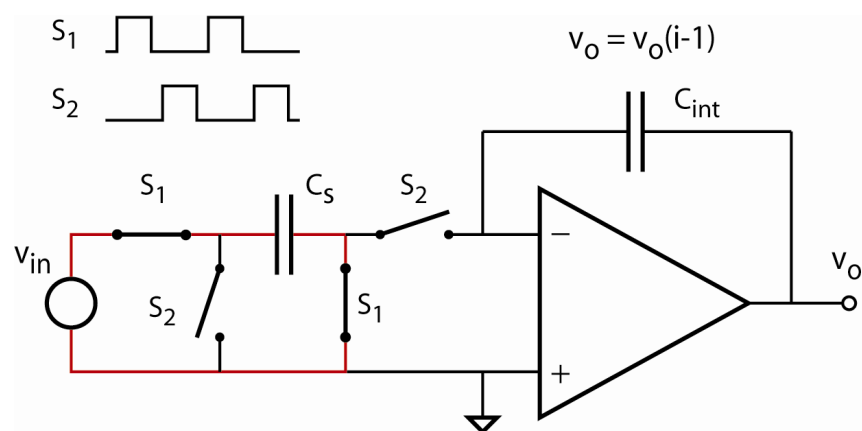


- For zero **accumulator input**
 $\Rightarrow w_i = \alpha w_{i-1}$, where $\alpha < 1$
- So, $w_1 = \alpha w_0$, $w_2 = \alpha^2 w_0$, ... $w_N = \alpha^N w_0$
 $\Rightarrow \mathbf{w}$ tends to zero

Exercise 1.4 (15min)

- What is the transfer function of a leaky accumulator?
- What is its DC gain?
- Run the m-file: MOD1_leak.m
 - This code simulates the behaviour of a 1st order modulator whose accumulator has a limited gain of 100
- What happens to the decimated output of the modulator?

Switched Capacitor Accumulator (1)



- Input voltage is sampled on C_s

Frequency Domain Analysis (1)

- Examining the spectrum of the simulated output signal of a sigma-delta modulator provides extra insight.
- The FFT transforms an N point discrete time signal into an N point discrete frequency spectrum.

$$X(f) = \sum_{n=0}^{N-1} \{ x(n) \cdot \exp(-j2\pi f n) \}, \quad f = 0, 1/N, 2/N \dots (N-1)/N.$$

- Note that f is the **normalized** sampling frequency (= freq/Fs)
- The N frequency points are known as the **FFT bins**

Frequency Domain Analysis (2)

- The FFT can be easily computed in Matlab

```
N = (1:100);
X = cos(2*pi*N/100);
FFTX = abs(fft(X)); % this is the magnitude spectrum
plot(FFTX);          % note: first term is the DC term
```

- For real signals we only need to compute the FFT from 0 to Fs/2
- The FFT resolution is the frequency difference between two neighboring discrete frequency points = Fs/N.
- More samples $\Rightarrow$ More FFT resolution

Exercise 1.5 (5min)

- Run the m-file: "fdomain.m"
- This computes the FFT of a sine wave sampled at F_s .
- It then plots the resulting magnitude spectrum from 0 to $F_s/2$

- Change the number of samples.
 - What happens to the FFT? Why?
- Vary the amplitude of the signal
 - What happens to the FFT? Is it equal to "ain"?
- Change the frequency to an irrational number such as $1/\pi$ kHz
 - What happens to the shape of the FFT? Why?

Windowing

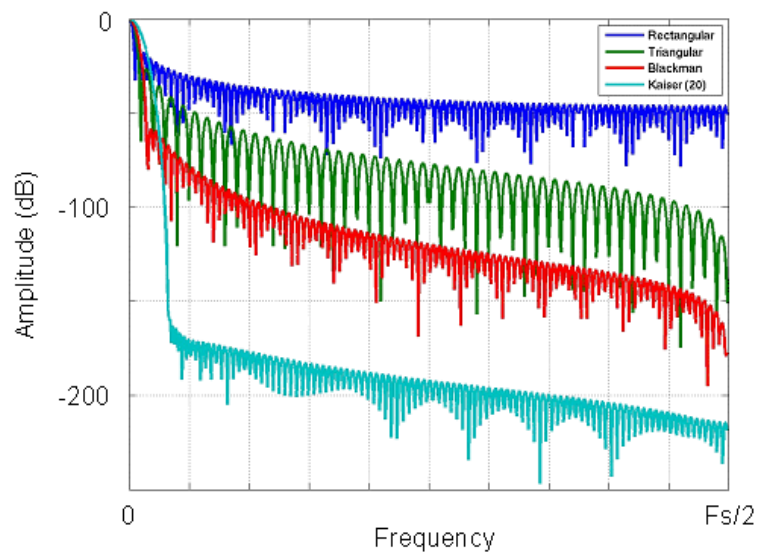
- If f_{in} does not fall exactly into a FFT bin, i.e. $f_{in} = F_s/2\pi$ then some of its energy will "leak" into neighboring bins

- Minimizing leakage $\Rightarrow$ multiplying input by a window function
- Note: Doing nothing $\Rightarrow$ rectangular window $\Rightarrow$ sinc(f) response

- Computed FFT is the convolution of the window function's frequency response with the actual input spectrum
- So the ideal window function should have a narrow bandwidth and suppress high frequencies

- Using fdomain.m, see what happens if you apply the Hann window to the signal x

Digital filter: Frequency domain (n=200)



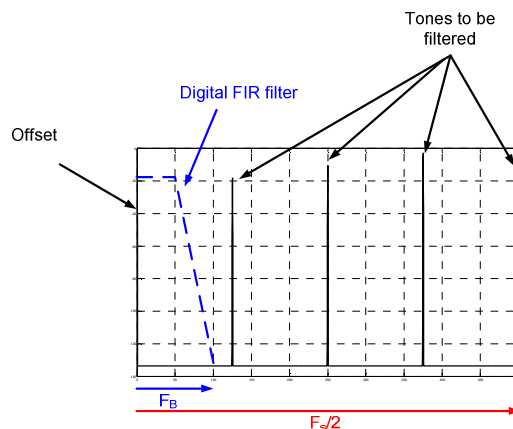
Copyright ©2008 Lucien Breems 109

Exercise 1.6 (10min)

- Run the matlab m-file: "MOD1_fft.m"
- This m-file runs MOD1.m, but applies an FFT function to the output bit-stream, and plots the spectrum from 0 to $f_s/2$.
- An input offset of 0 is applied, and at the output spectrum a single idle tone at $f_s/2$ is visible.
- Try input values of 0.25V, 0.5V, 50mV, and 34.6mV as the modulator input and observe the tones in the modulator output spectrum.

SNR Calculation (1)

- $\text{SNR} = \text{Signal Power} / \text{Noise Power (in the band of interest)}$



SNR Calculation (2)

- The bandwidth of the filter should be set such that the unwanted tones do not appear in its band, as this will decrease the modulator's calculated SNR
- To calculate the SNR, the FFT resolution (number of bins) must be taken into account
- The **signal power** is the sum of the energy in the bins related to the signal, while the energy in the rest of the bins is related to the noise and is integrated up to the system bandwidth.
- The resulting ratio (the SNR) can then be calculated.

Exercise 1.7 (5min)

- Run the Matlab file "MOD1_SNR.m"
- This file runs the single-bit modulator and applies a single tone sinusoid at its input.
- The sampling rate is 1000, and the signal frequency is at 2.
- A bandwidth of 20 is considered for SNR calculation.
- The m-file plots the output spectrum of the modulator, and highlights the in-band and signal bins.
- The energy in the signal and noise bins is calculated and their difference appears as SNR in the Matlab command prompt.

FFT Scaling

- Scaling does NOT affect the calculated SNR!
It only affects the meaning of 0dB
- Simple scaling e.g. $\text{fft}/\max(\text{fft}) \Rightarrow \text{maximum value} = 0\text{dB}$
- We can also scale so that 0dB corresponds to $A=1$
- The N-point fft of a sinewave will have a peak value of $A*N/2$
 - **IF** it falls exactly in a bin
 - and **IF** there is no windowing
- With a window, we have to multiply by a window-specific scale factor (see Schreier and Temes)